

Les arbres

October 5, 2026

1 Introduction

Dans une structure linéaire, les données sont mises les unes après les autres : chaque élément a au plus un seul prédécesseur et un seul successeur; Ce n'est pas toujours le meilleur choix. Dans un ordinateur, par exemple, un dossier contient des sous-dossiers, qui contiennent eux-mêmes des fichiers ou d'autres sous-dossiers. Ici, un élément a toujours un seul prédécesseur, mais il peut avoir plusieurs successeurs. Pour trouver un fichier, on suit un chemin (*e.g.* */Dossiers/Livres/q.pdf*), plutôt que de tout parcourir, élément par élément. Les éléments ne sont donc pas structurés linéairement, mais hiérarchiquement; Ce qui construit une *arborescence*.

2 Définitions et vocabulaire de base

Définition

Un arbre est une structure hiérarchique composée de nœuds reliés par des arêtes : à l'origine on trouve la racine, puis chaque nœud lié à exactement un seul nœud père et ses nœuds fils, ainsi de suite jusqu'à atteindre les feuilles, les nœuds sans fils.

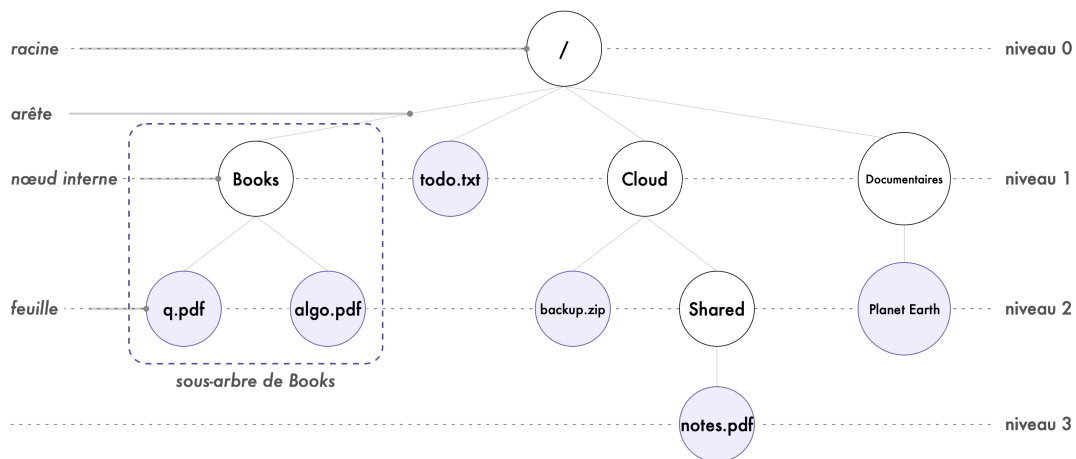
Vocabulaire de base

- **Racine** : l'entrée de l'arbre.
- **Nœud interne** : un nœud ayant au moins un fils.
- **Feuille** : un nœud sans fils.
- **Arête** : le lien entre père et fils.
- **Sous-arbre** : un nœud et tous ses descendants.
- **Profondeur d'un nœud** : nombre d'arêtes entre la racine et ce nœud.
- **Hauteur d'un arbre** : profondeur maximale parmi tous les nœuds.
- **Arité (degré) d'un nœud** : nombre de ses fils.
- **Arité d'un arbre** : arité maximale qu'un nœud peut avoir.
- **Taille d'un arbre** : nombre total de nœuds.

Par convention, on dessine un arbre à l'envers : la racine en haut, les feuilles en bas.

3 Exemple

On reprend l'exemple d'une arborescence de dossiers et de fichiers :



4 Arbre binaire

4.1 Définition

Un arbre binaire est un arbre où chaque nœud a au plus deux fils : un fils gauche et un fils droit. Récursivement, l'arbre il est soit :

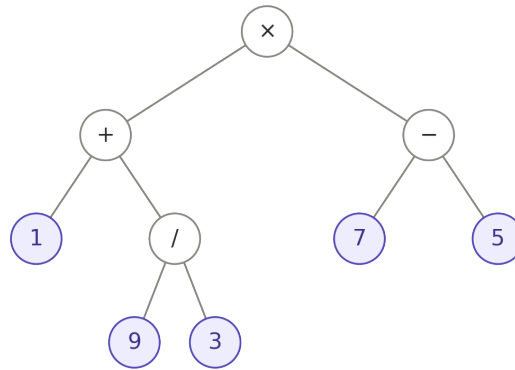
- vide ;
- formé d'une racine portant une valeur, et de deux arbres binaires : le sous-arbre gauche et le sous-arbre droit.

Attention : La place compte. Même avec un seul fils, il faut préciser s'il est à gauche ou à droite.

4.2 Exemple

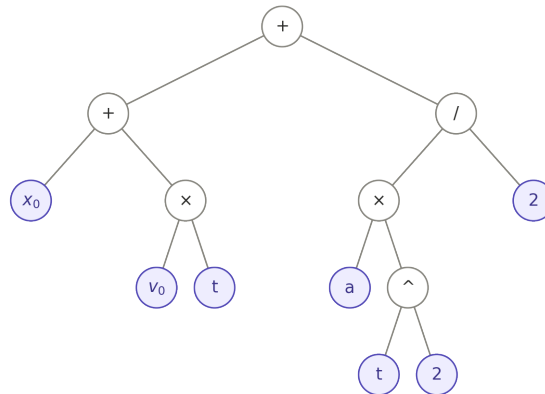
Une expression arithmétique peut être représenté par un arbre binaire où les nœuds internes sont les opérateurs (chacun a deux opérandes : sous-arbre gauche et sous-arbre droit) et les feuilles sont les nombres ou les variables. La forme de l'arbre donne la priorité des opérations, donc aucune parenthèse n'est nécessaire, l'opérateur appliqué en dernier lors de l'évaluation doit être à la racine et on calcule de bas en haut.

Soit l'expression $(1 + 9/3) \times (7 - 5)$, sa représentation arborescente est :



4.3 Exercice

Donner la représentation arborescente de l'expression arithmétique $x_0 + v_0 t + \frac{at^2}{2} (*)$.



4.4 Représentation en Python

4.4.1 L'idée

En utilisant des listes imbriquées :

- un arbre vide est la liste vide [];
- un nœud est une liste à 3 éléments : [valeur, sous_arbre_gauche, sous_arbre_droit];
- une feuille est donc [valeur, [], []] (ses deux sous-arbres sont vides).

4.4.2 Exemple

L'expression arithmétique vue précédemment peut être représentée comme suit :

$$\left[\underbrace{['x']}_{\text{valeur}}, \underbrace{['+', [1, [], []], ['/', [9, [], []], [3, [], []]]}_{\text{sous-arbre gauche}}, \underbrace{['-', [7, [], []], [5, [], []]]}_{\text{sous-arbre droit}} \right]$$

4.4.3 Exercice

Donner la représentation de l'expression (*).

```
[14]: expression = ['x',  
    ['+', [1, [], []], ['/', [9, [], []], [3, [], []]]],  
    ['-', [7, [], []], [5, [], []]]  
]
```

Pour accéder aux éléments : l'indice 0 donne la valeur de la racine, 1 le sous-arbre gauche et 2 le sous-arbre droit.

```
[4]: # valeur de la racine  
expression[0]
```

```
[4]: 'x'
```

```
[2]: # sous-arbre gauche  
expression[1]
```

```
[2]: ['+', [1, [], []], ['/', [9, [], []], [3, [], []]]]
```

```
[3]: # sous-arbre droit  
expression[2]
```

```
[3]: ['-', [7, [], []], [5, [], []]]
```

```
[5]: # valeur de la racine du sous-arbre gauche  
expression[1][0]
```

```
[5]: '+'
```

```
[6]: # valeur de la racine du sous-arbre droit  
expression[2][0]
```

```
[6]: '-'
```

4.4.4 Fonctions de base

Pour ne pas manipuler les indices partout, on écrit quelques petites fonctions :

```
[7]: # créer un arbre vide  
def arbre_vide():  
    return []
```

```

# créer un arbre à partir d'une valeur et deux sous-arbres
def arbre(v, fg, fd):
    return [v, fg, fd]

# vérifier si un arbre est vide
def est_vide(arbre):
    return arbre == []

# obtenir la valeur d'un arbre
def valeur(arbre):
    return arbre[0]

# obtenir le sous-arbre gauche; par convention renvoyer [] si l'arbre est vide
def fils_gauche(arbre):
    return [] if est_vide(arbre) else arbre[1]

# obtenir le sous-arbre droit
def fils_droit(arbre):
    return [] if est_vide(arbre) else arbre[2]

# vérifier si un arbre est une feuille
def est_feuille(arbre):
    return not est_vide(arbre) and est_vide(arbre[1]) and est_vide(arbre[2])

# vérifier si un arbre est un nœud interne
def est_interne(arbre):
    return not est_vide(arbre) and not est_feuille(arbre)

```

Un arbre est défini à partir d'arbres plus petits (ses sous-arbres). Les fonctions sur les arbres s'écrivent donc naturellement récursivement :

- **Cas de base** : l'arbre est vide, on renvoie un résultat simple ;
- **Cas général** : on appelle la fonction sur le sous-arbre gauche et sur le sous-arbre droit, puis on combine les résultats avec la racine.

```
[19]: # calculer la taille d'un arbre
```

```

def taille(arbre):
    if est_vide(arbre):
        return 0
    return 1 + taille(fils_gauche(arbre)) + taille(fils_droit(arbre))

```

```
[22]: taille(expression)
```

```
[22]: 9
```

```
[24]: # calculer la hauteur d'un arbre;
# par convention un arbre vide est de hauteur -1, pour qu'une feuille soit de
↳ hauteur 0
```

```
def hauteur(arbre):
    if est_vide(arbre):
        return -1
    return 1 + max(hauteur(fils_gauche(arbre)), hauteur(fils_droit(arbre)))
```

```
[25]: hauteur(expression)
```

```
[25]: 3
```

```
[21]: # calculer le nombre de feuilles

def nombre_feuilles(arbre):
    if est_vide(arbre):
        return 0
    if est_feuille(arbre):
        return 1
    return nombre_feuilles(fils_gauche(arbre)) +
↳ nombre_feuilles(fils_droit(arbre))
```

```
[26]: nombre_feuilles(expression)
```

```
[26]: 5
```

5 Parcours d'arbre

Un parcours est un algorithme qui visite chaque nœud exactement une fois; On trouve deux familles : le parcours en *profondeur* (on va au bout d'une branche avant de passer à la suivante) et le parcours en *largeur* (on explore niveau par niveau).

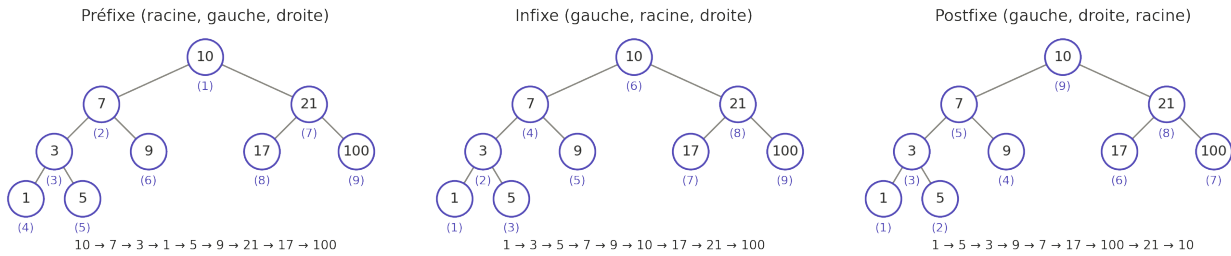
5.1 Parcours en profondeur

Il y a trois variantes. La seule différence est le moment où l'on traite la racine par rapport à ses deux sous-arbres :

- *Préfixe* : racine $\rightarrow$ sous-arbre gauche $\rightarrow$ sous-arbre droit.
- *Infixe* : sous-arbre gauche $\rightarrow$ racine $\rightarrow$ sous-arbre droit.
- *Suffixe* : sous-arbre gauche $\rightarrow$ sous-arbre droit $\rightarrow$ racine.

5.1.1 Exemple

Voici le même arbre parcouru de trois façons. Le petit numéro indique l'ordre de visite :



Généralement, la fonction de chaque parcours est récursive ; Le seul changement est la position du traitement de la racine en cours par rapport aux deux appels récursifs.

On prend comme traitement, par exemple, l'ajout de la valeur de la racine en cours à une liste ; Dans ce cas les trois fonctions sont :

```
[2]: def préfixe(arbre):
      if est_vide(arbre):
          return []
      return [valeur(arbre)] + préfixe(fils_gauche(arbre)) +
      ↪préfixe(fils_droit(arbre))
```

```
[3]: def infixe(arbre):
      if est_vide(arbre):
          return []
      return infixe(fils_gauche(arbre)) + [valeur(arbre)] +
      ↪infixe(fils_droit(arbre))
```

```
[4]: def suffixe(arbre):
      if est_vide(arbre):
          return []
      return suffixe(fils_gauche(arbre)) + suffixe(fils_droit(arbre)) +
      ↪[val(arbre)]
```

On reprend la représentation de l'arbre ci-dessus :

```
[31]: nombres = [10, [7, [3, [1, [], []], [5, [], []]], [9, [], []]], [21, [17, [],
      ↪[]], [100, [], []]]
```

```
[32]: # son parcours préfixe
      préfixe(nombres)
```

```
[32]: [10, 7, 3, 1, 5, 9, 21, 17, 100]
```

```
[33]: # son parcours infixe
      infixe(nombres)
```

```
[33]: [1, 3, 5, 7, 9, 10, 17, 21, 100]
```

```
[35]: # son parcours suffixe
```

```
suffixe(nombres)
```

```
[35]: [1, 5, 3, 9, 7, 17, 100, 21, 10]
```

5.2 Parcours en largeur

On visite d'abord la racine, niveau 0, puis tous les nœuds du niveau 1 de gauche à droite, puis ceux du niveau 2, etc. Pour cela, on utilise une file d'attente : le premier nœud entré est le premier sorti :

- Au début, on place la racine dans la file ;
- Tant que la file n'est pas vide : on retire le premier nœud, on le traite, puis on ajoute à la fin de la file son fils gauche puis son fils droit (s'ils ne sont pas vides) ;
- Quand la file est vide, tous les nœuds ont été visités.

```
[8]: def largeur(arbre):
```

```
    r = []
```

```
    file = []
```

```
    if arbre:
```

```
        file.append(arbre)
```

```
    while file:
```

```
        x = file.pop(0)
```

```
        r.append(x[0])
```

```
        if x[1]:
```

```
            file.append(x[1])
```

```
        if x[2]:
```

```
            file.append(x[2])
```

```
    return r
```

```
a0 = [0, [1, [3, [], []], [4, [], []]], [2, [5, [], []], [6, [], []]]]
```

```
largeur(a0)
```

```
[8]: [0, 1, 2, 3, 4, 5, 6]
```

```
[13]: def infixe2(arbre):
```

```
    if est_vide(arbre):
```

```
        return ''
```

```
    if est_feuille(arbre):
```

```
        return str(arbre[0])
```

```
    return eval(str(infixe2(fils_gauche(arbre))) + valeur(arbre) +
```

```
↳ str(infixe2(fils_droit(arbre))))
```

```
infixe2(expression)
```

```
[13]: 8.0
```